

14. Ejercicio de CoT
BookingCalendar

Jonathan Ricardo Serna Galina

20074902

14. Ejercicio de CoT: BookingCalendar

Sistema para reservar salas de reuniones durante un mismo día. Registra la hora de inicio, la hora de fin y el nombre de la sala; evita conflictos, permite cancelar reservas y presenta los intervalos libres en una tabla.

Descripción del proyecto

La clase BookingCalendar concentra las reglas de negocio. main.py demuestra su funcionamiento en consola. Se utiliza únicamente la biblioteca estándar de Python, por lo que la ejecución no requiere instalar paquetes.

La actividad corresponde a las capturas de BookingCalendar. El PDF del ejercicio 11 se toma como referencia de formato: portada, descripción, código, ejecución y liga. El código fue generado con asistencia de IA a partir de la solicitud del estudiante.

Reglas y criterios de aceptación

- Cada reserva contiene inicio, fin y sala. book devuelve True al registrar y False ante un conflicto o datos inválidos.
- El traslape se verifica únicamente dentro de la misma sala. Se permiten reservas consecutivas y reservas simultáneas en salas diferentes.
- cancel devuelve True solo al eliminar una reserva con horario y sala exactos; en otro caso devuelve False.
- get_available_slots imprime la tabla Sala / Inicio / Fin y devuelve las filas para facilitar pruebas y reutilización.
- Las horas se muestran en formato HH:MM de 24 horas. Se aceptan enteros para mantener compatibilidad con las pruebas del docente.
- La jornada considerada es 00:00 a 24:00. No se permiten intervalos vacíos, invertidos o que crucen medianoche.

Diagrama de clases

BookingCalendar

```
_bookings: dict[str, list[tuple[int, int]]]
```

```
+ book(start, end, room) -> bool  
+ cancel(start, end, room) -> bool  
+ get_available_slots(room=None) -> list  
- _validate(start, end, room) -> tuple  
- _to_minutes(value) -> int  
- _format_time(minutes) -> str
```

El signo + representa métodos públicos y el signo - detalles internos. main.py utiliza una instancia de BookingCalendar; los archivos de pruebas crean instancias independientes.

Estructura y funcionamiento

Las reservas se almacenan en un diccionario: cada clave es una sala y cada valor es una lista ordenada de pares de minutos. Convertir las horas a minutos permite comparar, por ejemplo, 09:15 y 10:30 sin ambigüedad.

Dos intervalos se superponen cuando $\text{inicio_nuevo} < \text{fin_existente}$ y $\text{inicio_existente} < \text{fin_nuevo}$. Usar desigualdades estrictas permite que una reunión comience justo cuando termina otra.

Para calcular la disponibilidad se recorren las reservas ordenadas desde 00:00. Se agregan los espacios anteriores a cada reserva y, si corresponde, el espacio final hasta 24:00.

Alcance

Los datos permanecen en memoria durante la ejecución. No se incluyen fechas, usuarios, almacenamiento persistente ni interfaz Streamlit, porque la consigna de las capturas solicita la clase y su comportamiento en un día.

Código

Los siguientes listados reproducen los archivos completos. Los números de línea del PDF son únicamente referencias y no forman parte del código.

booking_calendar.py

```
1  """Reservas de salas para un día, sin dependencias externas."""
2
3  import re
4
5
6  class BookingCalendar:
7      """Administra intervalos [inicio, fin) independientes por sala.
8
9      Acepta horas enteras o cadenas HH:MM. 24:00 solo puede ser fin.
10     Las salas distinguen mayúsculas y eliminan espacios exteriores.
11     """
12
13     def __init__(self):
14         self._bookings = {}
15
16     @staticmethod
17     def _to_minutes(value):
18         """Convierte una hora válida a minutos desde medianoche."""
19         if type(value) is int and 0 <= value <= 24:
20             return value * 60
21         if isinstance(value, str) and re.fullmatch(r"\d{2}:\d{2}", value):
22             hour, minute = map(int, value.split(":"))
23             if 0 <= hour < 24 and 0 <= minute < 60:
24                 return hour * 60 + minute
25             if hour == 24 and minute == 0:
26                 return 1440
27             raise ValueError("Usa horas enteras de 0 a 24 o el formato HH:MM.")
28
29     @staticmethod
30     def _format_time(minutes):
31         """Devuelve una hora de 24 horas con minutos."""
32         return f"{minutes // 60:02d}:{minutes % 60:02d}"
33
34     def _validate(self, start, end, room):
35         """Valida el intervalo y el nombre antes de modificar datos."""
36         if not isinstance(room, str) or not room.strip():
37             raise ValueError("El nombre de la sala no puede estar vacío.")
38         start = self._to_minutes(start)
39         end = self._to_minutes(end)
40         if not 0 <= start < end <= 1440:
41             raise ValueError("El inicio debe ser anterior al fin del día.")
42         return start, end, room.strip()
43
44     def book(self, start, end, room):
45         """Reserva si los datos son válidos y no existe un traslape."""
46         try:
47             start, end, room = self._validate(start, end, room)
48         except ValueError:
49             return False
50         bookings = self._bookings.get(room, [])
51         if any(start < old_end and old_start < end
52               for old_start, old_end in bookings):
53             return False
54         self._bookings.setdefault(room, []).append((start, end))
```

booking_calendar.py (continuación 1)

```
55         self._bookings[room].sort()
56         return True
57
58     def cancel(self, start, end, room):
59         """Cancela únicamente una coincidencia exacta de sala y horario."""
60         try:
61             start, end, room = self._validate(start, end, room)
62         except ValueError:
63             return False
64         bookings = self._bookings.get(room, [])
65         if (start, end) not in bookings:
66             return False
67         bookings.remove((start, end))
68         return True
69
70     def get_available_slots(self, room=None):
71         """Imprime una tabla y devuelve filas con sala, inicio y fin.
72
73         Sin argumento consulta salas registradas. Una sala nueva que se
74         consulte explícitamente aparece libre durante todo el día.
75         """
76         if room is not None:
77             if not isinstance(room, str) or not room.strip():
78                 raise ValueError("Indica un nombre de sala válido.")
79             rooms = [room.strip()]
80         else:
81             rooms = sorted(self._bookings)
82         rows = []
83         for name in rooms:
84             cursor = 0
85             for start, end in self._bookings.get(name, []):
86                 if cursor < start:
87                     rows.append((name, self._format_time(cursor),
88                                 self._format_time(start)))
89                     cursor = end
90                 if cursor < 1440:
91                     rows.append((name, self._format_time(cursor), "24:00"))
92         width = max([4] + [len(name) for name in rooms])
93         print(f"{'Sala':<{width}} | Inicio | Fin")
94         print("-" * (width + 17))
95         for name, start, end in rows:
96             print(f"{name:<{width}} | {start} | {end}")
97         if not rows:
98             print("Sin huecos libres o sin salas registradas.")
99         return rows
```

main.py

```
1  """Demostración reproducible de BookingCalendar."""
2
3  from booking_calendar import BookingCalendar
4
5
6  def main():
7      calendar = BookingCalendar()
8      operations = [
9          (8, 10, "Sala 1"),
10         (10, 12, "Sala 1"),
11         (9, 11, "Sala 1"),
12         (14, 16, "Sala 1"),
13         (16, 17, "Sala 2"),
14         (14, 14, "Sala 1"),
15         ("18:30", "19:15", "Sala 2"),
16     ]
17     for start, end, room in operations:
18         result = calendar.book(start, end, room)
19         print(f"Reservar {room} ({start}, {end}): {result}")
20     print("Cancelar inexistente:", calendar.cancel(9, 11, "Sala 3"))
21     print("Cancelar Sala 1 (14, 16):", calendar.cancel(14, 16, "Sala 1"))
22     print("\nDisponibilidad después de cancelar:")
23     calendar.get_available_slots()
24
25
26 if __name__ == "__main__":
27     main()
```

cot_test.py

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Developer: Miguel Jara Maldonado.
5  Creation Date: 2026-08-27.
6  Description: Unit Tests for the Chain of Thought Booking Calendar code.
7  """
8
9  from booking_calendar import BookingCalendar
10
11
12  def run_tests():
13      calendar = BookingCalendar()
14
15      assert calendar.book(9, 11, "a") is True, "Booking method is incorrect."
16      assert calendar.book(10, 12, "b") is True, "Adjacent blocks must be allowed."
17      assert calendar.book(11, 12, "b") is False, "Overlap must be rejected."
18
19      calendar.book(14, 14, "d") is True, "Zero-length bookings must be rejected."
20      calendar.book(14, 12, "d") is True, "Impossible bookings must be rejected."
21
22      assert (
23          calendar.cancel(9, 11, "f") is False
24      ), "Unexisting bookings should not be cancelled."
25      assert calendar.cancel(9, 11, "a") is True, "Existing bookings can be cancelled."
26
27      calendar.book(8, 10, "Sala 1")
28      calendar.book(10, 12, "Sala 1")
29      calendar.book(14, 16, "Sala 1")
30      calendar.book(16, 17, "Sala 2")
31      calendar.get_available_slots()
32      print("All tests passed.")
33
34
35  run_tests()
36
37  if __name__ == "__main__":
38      run_tests()
```

test_booking_calendar.py

```
1  """Pruebas adicionales que complementan el archivo del docente."""
2
3  import contextlib
4  import io
5  import unittest
6
7  from booking_calendar import BookingCalendar
8
9
10 class BookingCalendarTests(unittest.TestCase):
11     def setUp(self):
12         self.calendar = BookingCalendar()
13
14     def slots(self, room=None):
15         with contextlib.redirect_stdout(io.StringIO()):
16             return self.calendar.get_available_slots(room)
17
18     def test_overlap_variants(self):
19         for start, end in [(9, 11), (8, 10), (10, 12), (8, 12)]:
20             with self.subTest(start=start, end=end):
21                 calendar = BookingCalendar()
22                 self.assertTrue(calendar.book(9, 11, "A"))
23                 self.assertFalse(calendar.book(start, end, "A"))
24
25     def test_adjacency(self):
26         self.assertTrue(self.calendar.book(9, 11, "A"))
27         self.assertTrue(self.calendar.book(11, 12, "A"))
28         self.assertTrue(self.calendar.book(8, 9, "A"))
29
30     def test_independent_rooms(self):
31         self.assertTrue(self.calendar.book(9, 11, "A"))
32         self.assertTrue(self.calendar.book(9, 11, "B"))
33
34     def test_invalid_intervals(self):
35         for start, end in [(14, 14), (14, 12), (-1, 2), (23, 25),
36                             (True, 2), (1.5, 2), ("9:00", "10:00"),
37                             ("12:60", "13:00"), (24, 24), (None, 3)]:
38             with self.subTest(start=start, end=end):
39                 self.assertFalse(self.calendar.book(start, end, "A"))
40             self.assertEqual(self.slots(), [])
41
42     def test_invalid_rooms(self):
43         for room in ["", " ", None, 7]:
44             self.assertFalse(self.calendar.book(9, 10, room))
45
46     def test_minutes(self):
47         self.assertTrue(self.calendar.book("09:15", "10:30", "A"))
48         self.assertFalse(self.calendar.book("10:29", "11:00", "A"))
49         self.assertTrue(self.calendar.book("10:30", "11:00", "A"))
50
51     def test_cancel_exact(self):
52         self.calendar.book(9, 11, "A")
53         self.assertFalse(self.calendar.cancel(9, 10, "A"))
54         self.assertFalse(self.calendar.cancel(9, 11, "B"))
```

test_booking_calendar.py (continuación 1)

```
55         self.assertFalse(self.calendar.cancel(None, 11, "A"))
56         self.assertTrue(self.calendar.cancel(9, 11, "A"))
57         self.assertFalse(self.calendar.cancel(9, 11, "A"))
58         self.assertEqual(self.slots("A"), [("A", "00:00", "24:00")])
59
60     def test_sorted_gaps(self):
61         for start, end in [(14, 16), (10, 12), (8, 10)]:
62             self.calendar.book(start, end, "A")
63             self.assertEqual(self.slots("A"), [
64                 ("A", "00:00", "08:00"), ("A", "12:00", "14:00"),
65                 ("A", "16:00", "24:00"),
66             ])
67
68     def test_full_day(self):
69         self.assertTrue(self.calendar.book(0, "24:00", "A"))
70         self.assertEqual(self.slots("A"), [])
71
72     def test_unknown_room(self):
73         self.assertEqual(self.slots("Nueva"), [
74             ("Nueva", "00:00", "24:00")])
75         self.assertEqual(self.slots(), [])
76
77     def test_normalized_name(self):
78         self.calendar.book(9, 11, " A ")
79         self.assertFalse(self.calendar.book(9, 11, "A"))
80         self.assertTrue(self.calendar.cancel(9, 11, " A "))
81
82     def test_invalid_query(self):
83         with self.assertRaises(ValueError):
84             self.slots(" ")
85
86     def test_table_output(self):
87         output = io.StringIO()
88         with contextlib.redirect_stdout(output):
89             self.calendar.get_available_slots("A")
90             for text in ["Sala", "Inicio", "Fin", "00:00", "24:00"]:
91                 self.assertIn(text, output.getvalue())
92
93
94 if __name__ == "__main__":
95     unittest.main(verbosity=2)
```

Ejecución

Ejecuciones reales realizadas el 15 de septiembre de 2026. Desde la carpeta del ejercicio, ejecutar los siguientes comandos. En Windows puede utilizarse python en lugar de python3.

```
python3 main.py
python3 cot_test.py
python3 -m unittest -v test_booking_calendar.py
```

Demostración: salida real

```
Reservar Sala 1 (8, 10): True
Reservar Sala 1 (10, 12): True
Reservar Sala 1 (9, 11): False
Reservar Sala 1 (14, 16): True
Reservar Sala 2 (16, 17): True
Reservar Sala 1 (14, 14): False
Reservar Sala 2 (18:30, 19:15): True
Cancelar inexistente: False
Cancelar Sala 1 (14, 16): True
```

Disponibilidad después de cancelar:

Sala	Inicio	Fin
Sala 1	00:00	08:00
Sala 1	12:00	24:00
Sala 2	00:00	16:00
Sala 2	17:00	18:30
Sala 2	19:15	24:00

La tercera reserva se rechaza por traslape. La reserva 14–14 se rechaza por duración nula. Tras cancelar 14–16 de Sala 1, el intervalo libre de la tarde se extiende desde 12:00 hasta 24:00.

Pruebas del docente: salida real

```
Sala  | Inicio | Fin
-----
Sala 1 | 00:00 | 08:00
Sala 1 | 12:00 | 14:00
Sala 1 | 16:00 | 24:00
Sala 2 | 00:00 | 16:00
Sala 2 | 17:00 | 24:00
a      | 00:00 | 24:00
b      | 00:00 | 10:00
b      | 12:00 | 24:00
All tests passed.
Sala  | Inicio | Fin
-----
Sala 1 | 00:00 | 08:00
Sala 1 | 12:00 | 14:00
Sala 1 | 16:00 | 24:00
Sala 2 | 00:00 | 16:00
Sala 2 | 17:00 | 24:00
a      | 00:00 | 24:00
b      | 00:00 | 10:00
b      | 12:00 | 24:00
All tests passed.
```

El mensaje final aparece dos veces porque el archivo adjunto ejecuta `run_tests()` a nivel de módulo y vuelve a invocarlo bajo `__main__`. Se conserva sin modificaciones.

Observación sobre el archivo proporcionado

Las expresiones para `book(14, 14, "d")` y `book(14, 12, "d")` no incluyen `assert`; por tanto, no comprueban el resultado. Además, comparan con `True` aunque el texto indica rechazo. La solución devuelve `False` y las pruebas adicionales lo verifican explícitamente.

Pruebas adicionales y estilo

```
test_adjacency (adjacency) ... ok
test_cancel_exact (cancel_exact) ... ok
test_full_day (full_day) ... ok
test_independent_rooms (independent_rooms) ... ok
test_invalid_intervals (invalid_intervals) ... ok
test_invalid_query (invalid_query) ... ok
test_invalid_rooms (invalid_rooms) ... ok
test_minutes (minutes) ... ok
test_normalized_name (normalized_name) ... ok
test_overlap_variants (overlap_variants) ... ok
test_sorted_gaps (sorted_gaps) ... ok
test_table_output (table_output) ... ok
test_unknown_room (unknown_room) ... ok
```

```
-----
Ran 13 tests in 0.000s
```

```
OK
```

Resultado: 13 pruebas adicionales aprobadas. Los subcasos también comprueban distintas formas de traslape y entradas inválidas. La revisión pycodestyle de booking_calendar.py, main.py y test_booking_calendar.py terminó con código 0 y sin observaciones.

Reproducción de la revisión PEP 8

```
python3 -m pip install pycodestyle
python3 -m pycodestyle booking_calendar.py main.py \
    test_booking_calendar.py
```

pycodestyle es opcional para revisar el estilo; no es necesario para ejecutar el programa. El archivo original del docente se preserva y no se incluye en esta revisión de los archivos nuevos.

Conclusión

El programa cumple los contratos de reserva y cancelación, separa las salas y muestra huecos libres exactos. La ejecución y las pruebas aportan evidencia verificable de los casos válidos, los conflictos y los límites de horario.

Liga del código

[Abrir la carpeta del ejercicio 14 en GitHub](#)

Repositorio: ecojrsg/Programacion-Asistida

Carpeta: 14vo Ejercicio CoT BookingCalendar

Rama: main